

Detailed Report: Code Analysis of 3_task.R

July 10, 2026

1 Introduction

This report provides a detailed, line-by-line analysis of the 3_task.R file, which corresponds to Task 3 of the project. The objective of this script is to perform post-hoc exploration of the data by assigning each observation to one of the mixture components (clustering) using the Maximum A Posteriori (MAP) criterion. Finally, it analyzes how these cluster assignments vary depending on the hour of the day.

2 Line-by-Line Code Analysis

2.1 Extracting Posterior Means

```
1 # =====
2 # TASK 3: Post-hoc Exploration and MAP Assignment (Clustering)
3 # =====
4
5 cat("\nExecuting Task 3: MAP clustering and empirical fractions...\n")
6
7 samples_mat <- as.matrix(mcmc_samples)
8
9 # Calculate posterior means for pi, alpha, and beta
10 pi_mean <- rep(0, best_model)
11 alpha_mean <- rep(0, best_model)
12 beta_mean <- rep(0, best_model)
13
14 for (h in 1:best_model) {
15   pi_mean[h] <- mean(samples_mat[, paste0("pi[", h, "]")])
16   alpha_mean[h] <- mean(samples_mat[, paste0("alpha[", h, "]")])
17   beta_mean[h] <- mean(samples_mat[, paste0("beta[", h, "]")])
18 }
```

- **Lines 1-7:** Introductory comments, a console print, and the conversion of the MCMC samples from the best model into a matrix format (`samples_mat`).
- **Lines 9-12:** Three vectors (`pi_mean`, `alpha_mean`, `beta_mean`) are initialized to store the posterior mean estimates of the parameters for each component of the mixture model.
- **Lines 14-18:** A loop iterates over each component `h` (from 1 to `best_model`). For each parameter (π , α , β) of the specific component, the arithmetic mean across all MCMC iterations is calculated. These posterior means are point estimates that will be used for the subsequent clustering step.

2.2 Calculating Assignment Probabilities

```
20 # Calculate the probability of each observation belonging to each component
21 # P(Z_i = h | y_i) proportional to pi_h * Beta(y_i | alpha_h, beta_h)
22 N <- nrow(df)
23 prob_matrix <- matrix(0, nrow = N, ncol = best_model)
24
25 for (i in 1:N) {
26   for (h in 1:best_model) {
27     prob_matrix[i, h] <- pi_mean[h] * dbeta(df$load_factor[i], alpha_mean[h], beta_mean[h])
28   }
29 }
```

```

29 # Normalize to sum to 1
30 prob_matrix[i, ] <- prob_matrix[i, ] / sum(prob_matrix[i, ])
31 }

```

- **Lines 20-23:** The number of observations N is extracted from the original data frame `df`. A `prob_matrix` of dimensions $N \times H$ (observations by components) is initialized with zeros to store the probabilities of assignment.
- **Lines 25-26:** A nested loop iterates over each observation `i` and each mixture component `h`.
- **Line 27:** For a given observation y_i , the unnormalized posterior probability of belonging to component h is calculated. By Bayes' theorem, $P(Z_i = h | y_i) \propto P(Z_i = h) \cdot P(y_i | Z_i = h)$. This corresponds to the estimated weight π_h multiplied by the Beta density evaluated at y_i using the parameters α_h and β_h .
- **Line 30:** After calculating the unnormalized probabilities for all components for the i -th observation, they are divided by their sum. This normalizes the probabilities so that they correctly sum to 1 across all components for that specific observation.

2.3 MAP Clustering Assignment

```

33 # Assign each observation to the component with the maximum a posteriori (MAP)
    probability
34 df$map_cluster <- apply(prob_matrix, 1, which.max)
35
36 # Convert map_cluster to a factor for plotting
37 df$map_cluster <- as.factor(df$map_cluster)

```

- **Line 34:** The Maximum A Posteriori (MAP) rule is applied. The `apply` function with `MARGIN = 1` (row-wise) finds the index of the component that has the maximum probability for each observation using `which.max`. This integer is assigned as the cluster label in the new column `map_cluster` of the original data frame `df`.
- **Line 37:** The cluster labels are converted into a categorical `factor` type, which is necessary for discrete color mapping in visualizations.

2.4 Data Aggregation by Hour

```

39 # Calculate the empirical fraction of observations in each component as a function of
    hour
40 cat("Calculating empirical fractions and generating plot...\n")
41
42 # Use dplyr to group and summarize
43 library(dplyr)
44 fractions_df <- df %>%
45   group_by(hour, map_cluster) %>%
46   summarise(count = n(), .groups = "drop") %>%
47   group_by(hour) %>%
48   mutate(fraction = count / sum(count))

```

- **Lines 43-48:** The `dplyr` package is loaded to manipulate the data frame efficiently. A pipeline (`%>%`) is used to create `fractions_df`:
 - `group_by(hour, map_cluster)`: The data is grouped by both the hour of the day and the assigned cluster.
 - `summarise(count = n())`: It counts the number of observations falling into each specific combination of hour and cluster.
 - `group_by(hour)`: The grouping is changed to only group by hour.
 - `mutate(fraction = count / sum(count))`: It calculates the relative fraction of each cluster within that specific hour (so that the fractions sum to 1 for every hour).

2.5 Plotting the Empirical Fractions

```
50 # Create the plot
51 p_fractions <- ggplot(fractions_df, aes(x = hour, y = fraction, fill = map_cluster)) +
52   # Usiamo geom_col (bar chart) che gestisce nativamente i cluster mancanti in certe ore
   # senza sballare le somme
53   geom_col(width = 1, position = "fill", alpha = 0.85) +
54   theme_minimal() +
55   scale_x_continuous(breaks = seq(0, 23, by = 2)) +
56   labs(
57     title = "Empirical Fraction of MAP Clusters by Hour",
58     x = "Hour of the Day (0-23)",
59     y = "Fraction of Observations",
60     fill = "Cluster"
61   ) +
62   theme(
63     plot.title = element_text(face = "bold", size = 14),
64     axis.title = element_text(face = "bold"),
65     legend.position = "bottom"
66   )
67
68 # Display the plot
69 print(p_fractions)
70
71 # Save the plot explicitly
72 ggsave("images/cluster_fractions_hourly.png", plot = p_fractions, width = 8, height = 5,
73        dpi = 300)
74 cat("\nPlot salvato con successo come 'images/cluster_fractions_hourly.png' nella
75     cartella del progetto.\n")
```

- **Lines 51-66:** A `ggplot2` stacked bar chart (`geom_col`) is generated to visualize how cluster assignments change over the 24 hours of a day.
 - `position = "fill"`: This ensures that the bars are stacked and forced to stretch from 0 to 1, accurately representing proportions/fractions regardless of absolute counts.
 - The x-axis is customized to show breaks every 2 hours (`seq(0, 23, by = 2)`).
- **Lines 69-73:** The generated plot is printed to the view and exported as a high-resolution PNG file located at `images/cluster_fractions_hourly.png`.

3 Conclusion

This task effectively performs hard clustering on the observed data using the rigorous MAP criterion derived from the Bayesian estimation. By analyzing the resulting clusters temporally (hour by hour), it translates abstract statistical components into interpretable insights, revealing daily patterns in the load factor dynamics. The use of `dplyr` for aggregation and `ggplot2` for stacked visualization makes the subsequent analysis highly robust and clear.